L-Expr: ANTLR-Grammatik und Vorrangregeln

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Von klassischer CFG zur ANTLR-Grammatik

- ANTLR ist eine Java-Bibliothek
- Generiert aus einer Grammatik einen Lexer und LL(*)-Parser
- Eingabe-String plus Lexer/Parser $\rightarrow$ Parse-Tree mit Kontextklassen
- Zusätzlich: Klassen für Visitor-Pattern

Erinnerung: Konkrete Syntax für L-Expr

```
eform ::= '(' '+' eform eform+ ')' | '(' '-' eform eform+ ')' | INT
INT   ::= [0-9]+
---

eform ::= '(' '*' eform eform+ ')' | '(' '/' eform eform+ ')' | '(' '-' eform ')'

lexpr ::= eform EOF
```

ANTLR-Grammatik für L-Expr (Parser-Teil)

```
grammar Lispy;

lexpr       : eform EOF ;

eform       : atom                                  #atomExpr
            | '(' '-' eform ')'                     #unaryMinusExpr
            | '(' '+' eform eform+ ')'              #addExpr
            | '(' '-' eform eform+ ')'              #subExpr
            | '(' '*' eform eform+ ')'              #mulExpr
            | '(' '/' eform eform+ ')'              #divExpr
            ;

atom        : INT ;
```

- `lexpr` ist der Einstiegspunkt
- `eform` entspricht unseren Ausdrücken

ANTLR-Grammatik für L-Expr (Lexer-Teil)

```
...


INT          : [0-9]+ ;

COMMENT      : ';;' ~[\r\n]* -> skip ;
WS           : [ \t\r\n,]+ -> skip ;
```

- Token-Typen: `INT`, `'('`, `'-'`, ...
- Whitespace und Kommentare werden ignoriert

Arbeitsweise von ANTLR

- ANTLR generiert aus der Grammatik: Lexer, Parser, Visitor, ...
- Lexer konsumiert Input und erzeugt Tokenstrom
- Parser konsumiert Tokenstrom und erzeugt Parse-Tree

```
var source = "(+ 1 2)";

var input = CharStreams.fromString(source);
var lexer = new LispyLexer(input);
var tokens = new CommonTokenStream(lexer);

var parser = new LispyParser(tokens);
var tree = parser.lexpr();
```

Web: Lispy (Grammatik und Regeln, Eingabe und Parse-Tree)

Verhalten des Lexers

1. **Längster Match** (Test: Eingabe “foo42”)

```
INT  : [0-9]+ ;
ID   : [a-zA-Z][a-zA-Z0-9]* ;
```

2. **Reihenfolge** in Grammatik definiert Priorität (Test: Eingabe “for”)

```
FOR : 'for' ;
ID  : [a-z][a-zA-Z]* ;
```

3. **Implizite Token**: haben höchste Priorität (Test: Eingabe “0”)

```
foo  : '0' | NUM EOF ;

NUM  : [0-9]+ ;
WS   : [ \t\r\n]+ -> skip ;
```

Berücksichtigung von Präzedenz im Parser

- **konkrete Grammatik** (gestuft):

```
expr  ::= term ('+' term)*
term  ::= atom ('*' atom)*
atom  ::= '-' atom
        | INT
```

- **ANTLR**-Grammatik:

```
expr  : term ('+' term)* ;
term  : atom ('*' atom)* ;
atom  : '-' atom
      | INT
      ;
```

Direkte Linksrekursion und ANTLR4 - automatische Präzedenz

Aus der Grammatik mit gestuften Regeln zur Berücksichtigung von Präzedenz und (Links-) Assoziativität ...

```
expr  : term ('+' term)* ;
term  : atom ('*' atom)* ;
atom  : '-' atom
      | INT
      ;
```

... wird eine Grammatik mit linksrekursiven Alternativen, wo sich der Vorrang über die Reihenfolge der linksrekursiven Alternativen bestimmt (von oben nach unten):

```
expr  : expr '*' expr
      | expr '+' expr
      | '-' atom
      | INT
      ;
```

Wrap-up

- Lexer-Regeln beginnen mit **Großbuchstaben**
  - Längster Match gewinnt, bei Gleichstand: zuerst definierte Regel
  - Implizite Tokens (`'...'`) haben Vorrang vor expliziten Lexer-Regeln
- Parser-Regeln beginnen mit **Kleinbuchstaben**
  - Regeln können Token-Typen (Lexer-Regeln oder `'...'`) und andere Parser-Regeln kombinieren
  - Regeln können Alternativen haben (`|`) sowie optionale/mehrfache Teile (`?`, `*`, `+`)
  - Bei Mehrdeutigkeit: Vorrang für die erste passende Alternative
  - Startregel + `EOF` erzwingt, dass der **gesamte** Input konsumiert wird
  - ANTLR4 erlaubt **direkte** Linksrekursion (indirekte nicht)
- Vorrang und Assoziativität für Ausdrücke in ANTLR:
  - Gestufte Grammatik-Regeln (`expr` / `term` / `atom`)
  - Linksrekursive Regel mit geordneter Alternativenliste
  - Linksassoziativität ist Standard



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.